

CSE 4403: Algorithms

Lecture21

Greedy Algorithms

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

Proof of Correctness(1/3) : Setup

Let,

J = the set of jobs in an **optimal solution**

I = the set of jobs selected by the **greedy method**

We need to show that I and J have the same total profit.

If $I = J$, there is nothing to prove, so assume that $I \neq J$

Goal: show $\text{profit}(I) = \text{profit}(J)$, so I is optimal too

Proof of Correctness(2/5) : Why a Mismatch Must Exist

Fact: If a set of jobs can all be scheduled without missing a deadline, then any smaller subset of them can too because removing a job never makes scheduling harder.

J can't be missing something I has: If I contained every job of J plus extra profitable jobs, I would earn strictly more than J . But J was assumed optimal. Contradiction.

I can't be missing something J has: Greedy only skips a job when there's truly no room left for it. If J can fit that job in, greedy never turns down a job it could still fit.

So neither set can fully contain the other. There must be some job a that is only in I , and some job b that is only in J .

Proof of Correctness(3/5) : Aligning the Schedules

A job common to both I and J might sit in a different time slot in each schedule.

Let S_i be the schedule (slot assignment) for I , and S_j be the schedule for J . We can always rearrange S_i and S_j without changing either one's total profit so that every shared job sits in the same slot in both.

	Slot 1	Slot 2	Slot 3	Slot 4
S_i		Job a		
S_j				Job a

Proof of Correctness(3/5) : Aligning the Schedules

Example:

Job a is common to both I and J , scheduled in **slot 2 in I** but **slot 4 in J** . Move a to slot 4 in I as well.

- If some other job already occupies slot 4 in I , move that job to slot 2 instead.
Shifting a job to an earlier slot never breaks its own deadline, so this is always safe.
- If slot 4 in I is empty, a simply moves there directly.

Repeating this for every shared job produces two aligned schedules, S_i and S_j' , where every job common to I and J occupies the same slot in both.

	Slot 1	Slot 2	Slot 3	Slot 4
S_i		Job a		
S_j				Job a

Proof of Correctness(4/5) : Comparing Profits

Let a be the highest-profit job in I but not J . After aligning, let s be the slot where a sits in S_I , and let b be whichever job (if any) sits in that same slot s in S_J' .

Both placements are valid, so $s \leq \text{deadline}(a)$ (a sits there in I) and $s \leq \text{deadline}(b)$ (b sits there in J).

Greedy fills slots permanently. Once taken, a slot never frees up. So the set of occupied slots only grows over time.

Suppose $\text{profit}(b) > \text{profit}(a)$. Then greedy reaches b before a . Since slot s is free later, at a 's turn, it must already have been free earlier at b 's turn and $s \leq \text{deadline}(b)$, so greedy would have placed b there.

But $b \notin I$. Contradiction. So $\text{profit}(a) \geq \text{profit}(b)$.

Proof of Correctness(5/5) : The Swap & Conclusion

Since $\text{profit}(a) \geq \text{profit}(b)$, replacing b with a in slot s cannot reduce the total profit: $J' = J - \{b\} \cup \{a\}$ has profit no less than J .

Repeat this swap for every job that differs between I and J . Each swap keeps profit the same or increases it, and after enough swaps, J becomes identical to I .

Since profit never decreased along the way, and J was optimal to begin with: $\text{profit}(I) = \text{profit}(J)$. So I must be optimal too.

Complexity

Sorting jobs by profit

$O(n \log n)$

Slot search

A naive backward scan costs $O(d)$ per job in the worst case, that is $O(n \cdot d)$, $d = \min(n, \text{maxdeadline})$, so $O(n^2)$ in the worst case. A disjoint-set (union-find) speedup brings it down to nearly $O(n \log n)$.

$O(n \log n)$

Sort jobs by profit, descending

$O(n \cdot d)$

Naive slot search ($d = \text{max deadline}$)

$O(n \log n)$

With a union-find speedup

Interval Partitioning

Given n lectures, each with a start and finish time, schedule all of them using the minimum number of classrooms, so that no two lectures assigned to the same room overlap.

□ Given

Lectures with (start, finish) times

□ Goal

Minimize the number of classrooms used

□ Constraints

Two lectures in the same room
must not overlap in time

Define depth: the maximum number of lectures that are mutually overlapping at any single point in time.

Solution Idea (Naive)

1

Apply interval scheduling

Pick the maximum set of non-overlapping lectures for room 1.

2

Repeat on the rest

Apply interval scheduling again on the leftover lectures, and assign them to room 2.

3

Keep going

Continue opening new rooms until every lecture is assigned somewhere.

4

Will it work?

This seems reasonable but does it always use the minimum number of rooms?

Counter-Example : The Naive Idea Fails

Lectures: (3,7) (5,9) (8,11) (12,13) (10,14) (16,18) (13,20)

The naive room-by-room approach assigns:

Room 1 = {(3,7), (8,11), (12,13), (16,18)}

Room 2 = {(5,9), (10,14)}

Room 3 = {(13,20)} — 3 rooms used.

But at no point in time do more than 2 lectures overlap (depth = 2). Only 2 rooms are actually needed. That is, the naive approach wastes a room.

Correct Greedy Solution

1

Sort by start time

Order all lectures by start time, ascending.

2

Scan in order

Take each lecture in turn, earliest start first.

3

Reuse a free room

If some already-open room is free (its last lecture finished by now), assign the lecture there.

4

Open a new room

If no open room is free, allocate a brand-new room for this lecture.

Proof of Correctness(1/2) : Lower Bound

Claim:

For any set of lectures R ,
the number of classrooms required is at least the depth of R

Rooms needed $\geq \text{depth}(R)$

Proof: Any lectures that are mutually in conflict must be scheduled in different rooms.

That alone forces at least $\text{depth}(R)$ rooms to be used.

Proof of Correctness(2/2) : The Greedy Bound Is Tight

Let d be the depth of R .

Claim: The greedy algorithm uses exactly d rooms.

Proof:

Suppose, for contradiction, greedy used more than d rooms. Let j be the first lecture scheduled into room $d+1$.

Since lectures are processed in order of start time, there must already be d lectures underway at that moment, one occupying each of the other d rooms, all in conflict with j .

But that means $d+1$ lectures are mutually in conflict at once, contradicting the fact that the depth is only d .

So greedy never needs more than d rooms, matching the lower bound exactly.

Complexity

Sorting by start time

$O(n \log n)$

Room assignment

With a min-heap of each open room's last finish time, finding a free room (or confirming none exists) costs $O(\log n)$ per lecture. So, in worst case it will take $O(n \log n)$.

$O(n \log n)$

Sorting lectures by start time

$O(\log n)$

Per-lecture room lookup (min-heap)

$O(n \log n)$

Overall running time

Huffman Encoding

Given a long string of characters, find a way to encode each character into a binary code so that the total length of the encoded string is minimized.

❑ Given

Characters with known frequencies

❑ Goal

Minimize the total length of the encoded string

❑ Constraints

Codes must be prefix-free, no code is a prefix of another

Code lengths depend on frequency: common characters should get short codes, rare ones can get long codes.

Why Prefix-Free Matters

Try Encoding 1: $a = 0$, $b = 1$, $c = 01$

Now decode the message: 0101

Is it “a, b, a, b”? Or “c, c”? Both are valid under this scheme. The message is ambiguous.

The problem: The code for a (“0”) is a prefix of the code for c (“01”).

A prefix-free code guarantees every message decodes to exactly one answer.

Comparing Encodings

Text to encode: "abcc"

Encoding 1: a = 0, b = 10, c = 11 → encoded string 0101111 (7 bits)

Encoding 2: c = 0, a = 10, b = 11 → encoded string 101100 (6 bits)

Note:

- Both are prefix-free, but they don't cost the same.
- Huffman encoding finds the prefix-free encoding with the smallest possible total length.

What the Optimal Tree Looks Like

To be prefix-free, characters must sit only at the leaves of the binary tree. Internal nodes never represent a symbol themselves.

To minimize total length, the highest-frequency characters should sit closer to the root (shorter codes), and the lowest-frequency characters should sit deeper in the tree (longer codes).

The Greedy Algorithm

1

Build the leaves

Create a leaf node for each character, holding its frequency, then insert all into a min-priority queue (min-heap).

2

Extract the two smallest

Repeatedly remove the two nodes with the lowest frequency.

3

Merge into a parent

Create a **new internal node** whose frequency is their sum, make the two extracted nodes its children, insert it back into the heap.

4

Repeat until one node remains

That final node is the root of the Huffman tree. The left edges are 0, right edges are 1.

Worked Example — 6 Characters

Total Freq = 100

Char	Freq	Code	Bits	Weighted (freq×bits)
f	45	0	1	45
e	16	111	3	48
d	13	101	3	39
c	12	100	3	36
b	9	1101	4	36
a	5	1100	4	20

Optimality of Huffman Encoding(1/3) : Setup

Proof by induction on the number of symbols n :

Base case: $n \leq 2$ (with only 1 or 2 symbols, only one possible tree shape exists.)

Assume the claim holds for any sequence of $n - 1$ frequencies.

We show it then also holds for any sequence of n frequencies.

Goal: Show $\text{cost}(H) = \text{cost}(G)$, where H is an optimal tree and G is the tree our algorithm builds

Optimality of Huffman Encoding(2/3) : The Reduction

Let $f_1 \leq f_2 \leq \dots \leq f_n$ be the frequencies. There's an optimal tree H where the two smallest, f_1 and f_2 , are siblings (proved next). Our algorithm's tree G also makes them siblings, since it merges the two smallest frequencies first.

Remove f_1 and f_2 from both trees, and make their parent a new leaf with frequency (f_1+f_2) . This gives smaller trees H' and G' , each with $n - 1$ elements.

By the inductive hypothesis: $\text{cost}(H') = \text{cost}(G')$.

Since $\text{cost}(H) = \text{cost}(H') + f_1 + f_2$ and $\text{cost}(G) = \text{cost}(G') + f_1 + f_2$, it follows that $\text{cost}(H) = \text{cost}(G)$.

Optimality of Huffman Encoding(3/3) : Proof of the Claim

Claim: Some optimal tree has the two smallest-frequency symbols, w and y , as siblings.

Proof: Take any optimal tree. If w and y are already siblings, done. Otherwise, look at whoever currently sits next to w , exactly one of three cases holds:

1. **Some other symbol z is w 's sibling.** Since y has the second-smallest frequency, $\text{freq}(z) \geq \text{freq}(y)$. Swap z and y 's positions in the tree. This pairs w with y and costs no more than before, since y is no heavier than z .
2. **w has no sibling at all (an "only child").** Remove w 's parent and attach w directly to its grandparent. This moves w one level shallower, strictly *decreasing* the cost. That would contradict optimality, so this case can never actually occur.
3. **w 's sibling is a subtree containing some deeper leaf z .** Since $\text{freq}(w) \leq \text{freq}(z)$ but w sits shallower than z , swapping their positions puts the lighter symbol deeper strictly *decreasing* the cost. Again, this contradicts optimality, so this case can't occur either.

Huffman Tree Building Complexity

Building the initial heap

$O(n)$ to heapify n leaf nodes.

Merging

$n - 1$ extract-min(pop) / insert(push) operations, each costing $O(\log n)$.

$O(n \log n)$

$n-1$ merges, each a heap operation

$O(\log n)$

Cost per extract-min / insert

$n - 1$

Internal nodes created (merges)

Pseudocode

Main:

```
freq = COMPUTE-FREQUENCIES(text)
```

```
C = { NODE(char, f) for (char, f) in freq }
```

```
root = HUFFMAN(C)
```

```
codeTable = {}           // map: char -> binary string
```

```
ASSIGN-CODES(root, "", codeTable)
```

```
encoded = ""
```

```
for ch in text:
```

```
    encoded = encoded + codeTable[ch]
```

```
decoded = DECODE(root, encoded)
```

```
return codeTable, encoded, decoded
```

Step 1: Compute frequency of each character

Step 2: Build the Huffman tree

Step 3: Generate the prefix codes by traversing the tree

Step 4: Encode the original text using the code table

Step 5: Decode (walk the tree bit by bit)

Pseudocode

HUFFMAN(C):

// C = set of n characters, each with a frequency c.freq

$n = |C|$

Q = MIN-PRIORITY-QUEUE(C) // keyed by freq, built in $O(n)$

for i = 1 to n - 1:

 z = new NODE()

 x = EXTRACT-MIN(Q) // lowest-freq node

 y = EXTRACT-MIN(Q) // 2nd lowest-freq node

 z.left = x

 z.right = y

 z.freq = x.freq + y.freq

 INSERT(Q, z)

return EXTRACT-MIN(Q) // root of the Huffman tree

Pseudocode

ASSIGN-CODES(node, code, codeTable)

if node is a LEAF:

 codeTable[node.char] = (code == "" ? "0" : code) // only 1 distinct char

return

ASSIGN-CODES(node.left, code + "0", codeTable)

ASSIGN-CODES(node.right, code + "1", codeTable)

Pseudocode

DECODE(root, bitstring):

 result = ""

 node = root

 for bit in bitstring

 if bit == "0":

 node = node.left

 else:

 node = node.right

 if node is a LEAF:

 result = result + node.char

 node = root // restart at root for next character

return result